

EEE 419/591 – Summer 2026

Homework 4 – 591

Classical Bit Demodulator

[Same instructions as HW 1. Significant changes are highlighted in bold font]

Solve the following problem. **Read the instructions of the problem on the next page to know what you need to submit.** Submit your work to the designated link on Canvas. If you upload a second attempt to Canvas and it renames your file, ignore the renaming.

In this HW, only limited AI is allowed. Examples of allowed prompts on this HW:

- What is a greatest common divisor of two numbers?
- How do I write an if statement in python? Give me an example
- How do I check if a number is integer in python?
- I got this error message. How do I interpret it?

Examples of prohibited prompts on this HW:

- Write me a code that calculates the greatest common divisor of a list of numbers in python
- I ran this code. [PASTE CODE HERE]. I got this error message. [PASTE THE ERROR MESSAGE HERE]. How do I fix my code?

Do not forget to cite your AI chat if you used any. Check the Syllabus document for details.

Design a “Classical Bit Demodulator” that demodulates the signal received over a noisy wireless channel to estimate the transmitted bits. Start with the code studied in Lecture 4.1. Major changes will be necessary.

First – Background

Bits are transmitted over a noisy wireless channel in the form of a sinusoidal signal as follows:

- Logic 0 $\rightarrow x(t) = \cos(\omega t + 0)$ is transmitted
- Logic 1 $\rightarrow x(t) = \cos(\omega t + \pi)$ is transmitted

Note that the information bits are “modulated” (i.e. encoded/embedded) in the phase where 0 radians correspond to Logic 0, while π corresponds to Logic 1. The received signal $y(t)$ is, thus, a noisy version of the transmitted signal $x(t)$ and is given by

$$y(t) = x(t) + n(t)$$

where $n(t)$ is the noise. Since the receiver can only see the noisy version $y(t)$ but cannot see the clean version $x(t)$, which is its objective, it applies a demodulation technique like the one you learnt in lecture video 4.1 (aka “Classical Demodulator”) to estimate the signal’s phase and, hence, estimate the transmitted bit. Note that the demodulator needed for this problem is a simpler version of the one implemented in Lecture 4.1 since, unlike 4.1, the phase is the only unknown to the receiver in this problem while the amplitude and frequency are assumed to be known.

In this problem, you need to write a code that implements such a “Classical Demodulator”. You will run a Monte Carlo simulation over a large number of bits to estimate its performance measured in bit-error-rate (BER).

Second – Implementation

I- Signal Generation (Transmission)

1. Generate N_B random bits of 0’s and 1’s equally likely. N_B is the size of your dataset

2. For each bit i , generate its corresponding signal $x_i(t)$ which consists of N samples using the following equation (the time axis is sampled N times):

$$x_i(t) = \cos(\text{OMEGA} * t + \text{phase}_i)$$

3. Calculate the received signal $y_i(t)$ for each bit i as

$$y_i(t) = x_i(t) + n_i(t)$$

where $n_i(t)$ is a noise vector with mean 0 and standard deviation SIGMA . By the end of this step, you should have a matrix $N_B \times N$

4. Choose the following parameters
 - a. N_B : choose a suitable value that gives accurate BER
 - b. $N = 3$ (number of samples)
 - c. $\text{OMEGA} = 2 * \pi / N$ rad/sec
 - d. $\text{SIGMA} = \sqrt{0.5 / \text{SNR}}$.
 - e. $\text{SNR} = 1$ (this corresponds to 0 dB)

II- Decoding (Reception)

- The code in Lecture 4.1 estimates the frequency, amplitude and phase for a signal with N samples corresponding to a single bit. Modify it to estimate the phase only. The frequency is fixed to $1/N$ hertz
- Expand your code to work with N_B bits.
- Evaluate its performance by calculating the Bit-Error-Rate: $\text{BER} = \# \text{ bits in error} / N_B$.
- Note the following:
 - Your Classical Demodulator script should produce N_B bit estimates
 - You are, optionally, encouraged to avoid the loop in this step and replace it with matrix manipulation to speed up your code.
 - Make sure that the phase estimation part of your code can estimate the phase accurately. Hints:
 - Which command estimates the phase?
 - In which phase range does it work/fail?

Third – Deliverables

- One python file named “HW4_591.py” containing your entire code.
- Your code should print the BER value. DO NOT print anything else, not even the name of the variable
- Comment your plotting command, if any, before you submit your code
- Any precision is acceptable as long as it reveals at least 2 significant figures
- Optional – Not mandatory – Not extra-credit:
 - Share the BER value for with your classmates
 - Increase the SNR to a higher value and make sure the BER decreases. Plot a curve between the SNR on the x-axis (dB scale) and BER on the y-axis (log scale). Compare to the standard curves (Wikipedia). If you find mismatch, it is likely from the definition of SNR. Share the curve with your classmates.

MANDATORY – Reminder to cite AI chat – Check syllabus document for details